

SWE 580 Term Project Report

Tool Granularity in LLM-Based Knowledge Retrieval

Sena Oz

The starter code provides a 100-note markdown vault with a Whoosh search backend (10-result cap) and a working evaluator. The task is to design two different tool interfaces on top of this backend, evaluate them against 25 ground-truth queries, tune the prompts, and compare the results.

Main two configurations:

- **Config A** — 4 coarse-grained tools, combinable parameters
- **Config B** — 9 fine-grained tools, one operation each

The model used is google/gemini-2.5-flash via OpenRouter.

Config A — 4 Tools

Tool	What it does
search_notes	Full-text + tags + date range in one call
get_note	Retrieve by title or path
get_related_notes	Links in/out/both via direction param
get_vault_overview	Vault stats + 5 recent notes

Multi-faceted queries like "python notes before January 8" become a single call: `search_notes(tags: ["python"], date_to: "2026-01-07")`. Both filters are applied at the backend before the 10-result cap.

Config B — 9 Tools

Tool	What it does
search_by_content	Full-text search
search_by_tags	Filter by tags
search_by_date	Filter by date range
get_note_by_title	Get note by title
get_note_by_path	Get note by path
get_incoming_links	Backlinks to a note
get_outgoing_links	Links from a note

get_vault_stats	Vault statistics
get_recent_notes	Most recent notes

The same multi-faceted query requires two calls: `search_by_tags(["python"])` then `search_by_date(date_to: "2026-01-07")`, with the LLM intersecting results in its context window.

Key Structural Difference

With a 10-result cap, the intersection approach in Config B is vulnerable: if the target note ranks 11th in either individual search, it won't appear in either list and can't be intersected. Config A avoids this because both filters go to the backend together — the cap applies to the already-filtered result set.

Comparison

Prompt Tuning

Both configs started with a minimal 2-line system prompt. I diagnosed failures from the trace logs and tuned iteratively.

Config A tuning:

Version	Success	Matches	Avg Tokens	What changed
Baseline	40%	10/25	1,296	—
v1	84%	21/25	2,752	Output format, vault structure, folder/tag distinction, tool guide
v2	88%	22/25	3,497	Date boundary precision, folder query fallback pattern
v3	84%	21/25	3,718	Over-specified multi-hop — regression, reverted to v2

Config B tuning:

Version	Success	Matches	Avg Tokens	What changed
Baseline	24%	6/25	1,730	—
v1	84%	21/25	3,560	Output format, tool selection table, intersection patterns
v2	76%	19/25	3,810	Over-specified — regression, stayed with v1

The biggest single gain in both configs was adding the `RESULT_PATHS: [...]` output format. The LLM was finding the right answers at baseline but reporting them in prose — the evaluator couldn't parse any of it. That one line fixed 8–10 queries per config.

Head-to-Head (Best Versions)

Config A v2 vs Config B v1, Gemini 2.5 Flash:

Metric	Config A	Config B	Winner
Success rate	88% (22/25)	84% (21/25)	A
Avg tool calls	1.08	1.40	A
Avg tokens	3,377	3,857 (-14%)	A
Avg latency	1.541s	1.503s	B (marginal)

By Category

Category	Config A	Config B	Note
simple_lookup (2)	100%	100%	Tied
tag_search (2)	100%	100%	Tied
temporal (3)	100%	100%	Tied
content_search (2)	100%	100%	Tied
multi_faceted (9)	78%	67%	A wins
graph_based (7)	86%	86%	Tied

The entire accuracy difference comes from multi_faceted queries.

Query-Level Breakdown

Query	Category	A	B	Tools A	Tools B	Tokens A	Tokens B
q01	simple_lookup	✓	✓	1	1	3,079	3,383
q02	simple_lookup	✓	✓	1	1	3,028	3,332
q03	tag_search	✓	✓	1	1	2,999	3,299
q04	tag_search	✓	✓	1	1	3,669	3,967
q05	temporal	✓	✓	1	1	2,997	3,296
q06	temporal	✓	✓	1	1	3,229	3,527

q07	content_search	✓	✓	1	1	3,006	3,307
q08	content_search	✓	✓	1	1	2,816	3,115
q09	multi_faceted	✓	✓	1	2	3,075	4,263
q10	graph_based	✓	✓	1	1	3,681	3,964
q11	multi_faceted	✓	✗	1	2	3,144	4,734
q12	multi_faceted	✓	✓	1	2	2,936	4,226
q13	multi_faceted	✓	✓	1	1	3,723	4,025
q14	multi_faceted	✓	✓	1	1	3,568	3,849
q15	multi_faceted	✓	✗	1	2	3,322	4,600
q16	graph_based	✓	✓	1	1	3,079	3,404
q17	graph_based	✓	✓	1	2	3,461	3,828
q18	graph_based	✗	✗	1	2	3,453	3,181
q19	graph_based	✓	✓	1	2	3,255	3,613
q20	graph_based	✓	✓	1	1	3,694	3,977
q21	graph_based	✓	✓	3	3	5,244	5,681
q22	multi_faceted	✗	✗	1	2	3,633	4,900
q23	multi_faceted	✗	✓	1	1	3,589	3,608
q24	multi_faceted	✓	✓	1	1	3,213	3,512
q25	temporal	✓	✓	1	1	3,519	3,822

Where configs diverge:

- **q11** (tag+date): A ✓, B ✗ — search_by_tags(["python"]) hit the cap; expected notes ranked beyond position 10 in the per-tag list.
- **q15** (tag+content): A ✓, B ✗ — intersection of two capped searches excluded both expected notes from both lists.
- **q23** (folder+tag): A ✗, B ✓ — Config B used search_by_tags(["python", "reference"]) and found "reference" is a valid tag. Config A couldn't express this without folder/tag ambiguity.

- **q18, q22:** Both fail — multi-hop constraints neither config handled correctly regardless of tool design.

Findings

Finding 1: Output format is the most impactful prompt element

Adding `RESULT_PATHS`: [...] fixed more queries than any other change (8–10 per config in one step). At baseline, the LLM was reasoning correctly but outputting prose — nothing the evaluator could use. This is a non-obvious failure mode: the tool design was fine, the retrieval was correct, but the output contract was missing.

Finding 2: Coarse-grained tools are more token-efficient

Config A averaged 480 fewer tokens per query (14% less). The gap comes from multi-faceted queries where Config B uses 1.56 tool calls on average vs Config A's 1.00. Each extra call adds a full tool request + response round to the message history. For the 9 multi-faceted queries, Config B was ~26% more expensive in tokens.

Finding 3: Fine-grained tools have a naming advantage at baseline

Config B was worse overall at baseline (24% vs 40%) but better on temporal queries (67% vs 33%). `search_by_date` is immediately obvious; `search_notes` requires knowing it takes `date_from/date_to`. Similarly, `get_incoming_links` vs `get_outgoing_links` eliminates the direction ambiguity that `get_related_notes(direction: ...)` introduces. Fine-grained names are self-documenting in a way that reduces prompt guidance requirements.

Finding 4: Config B's intersection failure is structural, not fixable by prompting

q11 and q15 fail in Config B because the expected notes rank beyond position 10 in individual tag or content searches. No prompt change can fix this — the backend truncates before the LLM sees results. The fix would be to add combined-filter support at the backend (what Config A does), or raise the cap. This is a design-level issue, not a tuning issue.

Finding 5: Over-specification causes regressions

Config A v3 and Config B v2 both regressed after adding more detailed instructions. The LLM started applying rules to edge cases they weren't meant for. The best prompt is not the most complete one — it's the one that addresses actual observed failures without over-constraining unrelated queries.

Multi-Model Comparison

Same evaluation with google/gemini-2.0-flash-001:

	Gemini 2.5 Flash	Gemini 2.0 Flash
Config A success	88%	80%
Config B success	84%	80%

Config A avg tokens	3,377	4,585
Config B avg tokens	3,857	5,825
Token gap (B – A)	480	1,240

Config A's token efficiency advantage holds and grows with the weaker model (+14% with 2.5 Flash vs +27% with 2.0 Flash). The older model makes more tool calls per query, which amplifies the per-call token overhead in Config B.

Accuracy ties at 80% with the weaker model — Config A loses its 4pp edge because the model can't reliably follow the detailed combined-filter instructions. The naming advantage of Config B tools is more robust to model capability degradation.

I also attempted Llama 3.3 70B via Groq but it produced malformed tool call syntax (<function=get_note{...} instead of JSON), failing at parse time. The Groq free tier (100k tokens/day) ran out before I got complete results, so I excluded it.

Limitations

- 25 queries is a small test set; categories with 2 queries have no statistical power.
- Prompts were tuned by observing failures — baseline numbers (40%/24%) are more representative of real zero-shot performance than the tuned numbers.
- The 10-result cap is hardcoded. A higher cap would reduce Config B's intersection disadvantage.
- Only tested Gemini model family. Other families (GPT-4, Claude) may show different patterns.

Summary

Config A is better when:

- Queries combine multiple constraints (tag + date, tag + content)
- Token cost matters (14–27% cheaper for multi-faceted queries)
- You're willing to invest in prompt engineering

Config B is better when:

- Prototyping fast with minimal prompting (clearer tool names)
- Queries are mostly simple or single-dimension

The structural result cap issue makes Config B fundamentally weaker for multi-faceted queries regardless of how good the prompt is. That's the most actionable finding from this comparison.